



南開大學
Nankai University

计算机学院
并行程序设计实验报告

多线程编程实验

姓名：熊诚义

学号：2313310

专业：计算机科学与技术

目录

1 问题描述	3
1.1 期末研究报告的大问题	3
1.2 本次实验涉及的子问题	3
2 实验要求	3
2.1 基础要求	3
2.2 进阶要求	3
3 Pthread 并行化	4
3.1 设计思路	4
3.1.1 并行化分析目标	4
3.1.2 线程函数设计	4
3.2 算法实现	4
3.2.1 核心数据结构与线程函数	4
3.2.2 主控函数实现	5
3.2.3 关键算法亮点	6
3.3 复杂性分析	7
3.3.1 时间复杂度分析	7
3.3.2 空间复杂性分析	8
3.4 实验及结果分析	8
3.4.1 正确性验证	8
3.4.2 结果分析	9
4 OpenMP 并行化	9
4.1 设计思路	9
4.1.1 并行模式与问题分析	9
4.1.2 线程局部存储策略	9
4.1.3 工作划分与负载均衡	9
4.2 算法实现	9
4.2.1 基本并行框架	9
4.2.2 数据结构与工作划分	10
4.2.3 密码生成过程	10
4.2.4 结果合并与同步	10
4.3 复杂性分析	11
4.3.1 时间复杂度分析	11
4.3.2 空间复杂度分析	11
4.3.3 并行效率分析	12
4.4 实验及结果分析	12
4.4.1 正确性验证	12
4.4.2 结果分析	13

5	Pthread 与 OpenMP 性能差异	13
5.1	性能差异分析	13
5.2	编译优化与平台适应性差异	14
6	一些尝试	15
7	并行计算的适用性分析	17
8	Git 项目链接	17

1 问题描述

1.1 期末研究报告的大问题

口令猜测算法的并行化优化是本次期末研究的核心问题。具体目标是通过并行化技术（如 SIMD、多线程、MPI、CUDA）加速以下两个关键流程：

1. **PCFG 口令生成**：利用概率上下文无关文法模型，按概率降序生成口令猜测列表。需解决优先队列的并行访问、preterminal 的分布式生成等问题。
2. **MD5 哈希计算**：对生成的口令进行快速哈希运算，以模拟攻击者的实际破解流程。需通过数据级并行提升哈希计算吞吐量。

研究需实现并行算法，分析加速比，并探索不同并行技术的适用性与优化空间。

1.2 本次实验涉及的子问题

PCFG 算法的工作原理是将密码模式分解为多个段 (segment)，如字母段 L、数字段 D 和符号段 S。每个段还有长度属性，如 L8 代表 8 个字母。算法通过组合不同的段来生成预终结符 (PT)，例如 L8D2 表示 8 个字母后跟 2 个数字。

本次实验关键并行化点在于：对于给定的 PT，需要用具体值填充其最后一个 segment 来生成实际的密码猜测。举例来说，当处理 L8D2 这样的 PT 时：

- L8 已经实例化为具体字符串，如"password"
- 需要为 D2 填充所有可能值，如"12"、"23"、"01" 等
- 这个填充过程完全独立，可以高效并行化

2 实验要求

2.1 基础要求

- 使用 pthread 实现对两个猜测生成循环的并行化
- 确保正确性：使用 correctness_guess.cpp 验证，比对攻破率 (Cracked) 数值
- 保证并行性能：测量加速比

2.2 进阶要求

- 同时实现 pthread 和 OpenMP 版本，对比性能差异
- 研究编译优化对猜测和哈希过程的影响差异
- 尝试同时优化多线程猜测和 SIMD 哈希，实现整体加速
- 思考并行化的适用场景和优化策略

3 Pthread 并行化

3.1 设计思路

3.1.1 并行化分析目标

经过分析以及注释说明，我发现 PCFG 口令猜测算法中两个关键循环是理想的并行化目标：

1. 单段 PT 处理循环 - 直接遍历单个 segment 的所有可能值
2. 多段 PT 处理循环 - 为最后一个 segment 填充所有可能值，前缀已确定

这两个循环具有以下特点：

- 任务之间完全独立，无数据依赖
- 计算量大但粒度均匀
- 只需在结果收集阶段同步

3.1.2 线程函数设计

我设计了通用的线程处理函数 `generate_guesses_thread`，具有以下功能：

1. **参数解析与本地存储初始化：**将传入的 `void*` 类型参数转换为 `ThreadData` 结构体指针，创建本地向量存储该线程生成的密码猜测，避免频繁锁操作
2. **生成密码猜测：**只处理分配给该线程的索引范围 (`start_idx` 到 `end_idx`)，根据是单段 PT 还是多段 PT 采用不同的处理逻辑，对于单段 PT，直接使用 `segment` 中的值作为猜测，而对于多段 PT，则将前缀与当前 `segment` 值拼接形成新猜测
3. **安全合并结果：**使用互斥锁保护共享资源访问，批量将本地生成的猜测添加到共享结果变量，更新总猜测计数器，最后完成后释放互斥锁

这种设计体现了“批量同步”的并行优化思想，每个线程先独立完成计算，存入本地缓冲区，然后一次性获取锁进行结果合并，最大限度减少了线程同步开销，提高了并行效率。

3.2 算法实现

PCFG 密码猜测算法的 Pthread 并行实现主要聚焦于两个计算密集型循环的并行化，具体实现如下：

3.2.1 核心数据结构与线程函数

首先定义了线程函数和数据传递结构：

```
1 // 线程参数结构体
2 struct ThreadData {
3     int start_idx;           // 线程处理的起始索引
4     int end_idx;             // 线程处理的结束索引
```

```

5     segment* seg;           // 指向 segment 的指针
6     string prefix;         // 前缀字符串 (多段 PT 使用)
7     vector<string>* result_guesses; // 共享结果向量
8     pthread_mutex_t* mutex; // 互斥锁
9     int* total_count;      // 计数器
10 };
11
12 // 线程工作函数
13 void* generate_guesses_thread(void* arg) {
14     ThreadData* data = (ThreadData*)arg;
15     vector<string> local_guesses;
16
17     // 在本地向量中生成猜测
18     for (int i = data->start_idx; i < data->end_idx; i++) {
19         string guess;
20         if (data->prefix.empty()) {
21             guess = data->seg->ordered_values[i]; // 单段处理
22         } else {
23             guess = data->prefix + data->seg->ordered_values[i]; // 多段处理
24         }
25         local_guesses.push_back(guess);
26     }
27
28     // 批量合并结果
29     pthread_mutex_lock(data->mutex);
30     for (const string& g : local_guesses) {
31         data->result_guesses->push_back(g);
32     }
33     *data->total_count += local_guesses.size();
34     pthread_mutex_unlock(data->mutex);
35
36     return NULL;
37 }

```

3.2.2 主控函数实现

在 PriorityQueue::Generate(PT pt) 函数中实现了并行处理逻辑：

```

1 void PriorityQueue::Generate(PT pt) {
2     // 计算 PT 概率
3     CalProb(pt);
4

```

```

5      // 互斥锁和线程初始化
6      pthread_mutex_t mutex;
7      pthread_mutex_init(&mutex, NULL);
8      int num_threads = 8;
9
10     if (pt.content.size() == 1) {
11         // 单段 PT 处理
12         // 获取对应 segment 数据...
13
14         // 准备线程参数
15         pthread_t threads[num_threads];
16         ThreadData thread_data[num_threads];
17
18         // 计算工作分片
19         int items_per_thread = pt.max_indices[0] / num_threads;
20         int remainder = pt.max_indices[0] % num_threads;
21
22         // 创建线程
23         for (int t = 0; t < num_threads; t++) {
24             // 分配索引范围...
25             pthread_create(&threads[t], NULL, generate_guesses_thread,
26                           (void*)&thread_data[t]);
27         }
28
29         // 等待线程完成
30         for (int t = 0; t < num_threads; t++) {
31             pthread_join(threads[t], NULL);
32         }
33     } else {
34         // 多段 PT 处理
35         // 1. 处理前缀：计算除最后 segment 外的值...
36         // 2. 准备最后 segment 数据...
37         // 3. 并行处理最后 segment 的所有可能值...
38         // (实现逻辑与单段相似)
39     }
40
41     pthread_mutex_destroy(&mutex);
42 }

```

3.2.3 关键算法亮点

1. 工作分片策略：采用了精确的工作分割方法，基于以下公式：

```
1 // 基本分片大小
2 int items_per_thread = total_items / num_threads;
3 // 余数处理
4 int remainder = total_items % num_threads;
5
6 // 每个线程的索引计算
7 start_idx = t * items_per_thread + min(t, remainder);
8 end_idx = (t+1) * items_per_thread + min(t+1, remainder);
```

这确保了任务均匀分布，且每个项目只分配给一个线程。

2. **批处理与局部存储：**每个线程先在本地向量中积累所有结果，然后一次性获取锁进行合并，显著减少了锁竞争。
3. **资源管理：**正确初始化和销毁互斥锁，使用 `join` 等待所有线程完成，避免资源泄露。
4. **通用设计：**线程函数设计支持单段和多段 PT 的处理，通过 `prefix` 参数区分，提高代码复用性。

这种实现方案有效地将原本串行的密码猜测生成过程分解为多个并行任务，在保证正确性的同时提高了处理速度，特别是对于大型密码集的生成效率显著提升。

3.3 复杂性分析

3.3.1 时间复杂度分析

串行算法复杂度：

- 单段 PT 处理： $O(M)$ ，其中 M 是 segment 中值的数量
- 多段 PT 处理： $O(M)$ ，其中 M 是最后一个 segment 中值的数量
- 总体复杂度： $O(N*M)$ ，其中 N 是处理的 PT 的数量

并行算法的复杂度：

- 理论并行复杂度： $O(N*M/P)$ ， P 是线程数
- 每个线程处理： $O(M/P)$ 个项目
- 线程内部合并结果： $O(M/P)$
- 线程间同步开销： $O(P)$

考虑 Amdahl 定律，由于生成猜测的代码几乎 100% 可并行，加速比接近于线程数 P ，但受限于共享资源访问互斥限制、线程创建和销毁开销以及线程间负载不均衡。

3.3.2 空间复杂性分析

- **线程局部存储:** $O(M)$, 每个线程存储约 M/P 个局部结果
- **共享结果向量:** $O(N*M)$, 存储所有生成的密码猜测
- **线程管理结构:** $O(P)$, 线程句柄和参数结构
- **同步原语:** $O(1)$, 互斥锁

总空间复杂度: $O(N*M + P)$, 主要由结果存储决定

3.4 实验及结果分析

以下是串行版本与 Pthread 并行化版本正确性验证与 Guess time 对比:

不进行编译优化

Guess time:7.87898seconds	Guess time:8.28865seconds
Hash time:14.8155seconds	Hash time:16.4154seconds
Train time:95.5076seconds	Train time:97.4554seconds
Cracked:358217	Cracked:358217

图 3.1: 串行 (左) 与 Pthread (右) 数据对比

-O1 编译优化

Guess time:0.740384seconds	Guess time:0.694773seconds
Hash time:10.0195seconds	Hash time:8.86148seconds
Train time:42.1039seconds	Train time:30.1586seconds
Cracked:358217	Cracked:358217

图 3.2: 串行 (左) 与 Pthread (右) 数据对比

-O2 编译优化

Guess time:0.624728seconds	Guess time:0.864357seconds
Hash time:8.56184seconds	Hash time:7.9023seconds
Train time:27.5051seconds	Train time:27.806seconds
Cracked:358217	Cracked:358217

图 3.3: 串行 (左) 与 Pthread (右) 数据对比

3.4.1 正确性验证

由实验数据, Pthread 并行猜测算法的攻破率和串行猜测算法的攻破率 (Cracked) 大致相同 (完全相同...), 所以实现的 Pthread 并行化正确。

3.4.2 结果分析

由实验数据可发现, 我所实现的 Pthread 并行化仅在-O1 编译优化下千万级数量级口令下得到了相对串行算法的加速, 加速比为 **1.06565**, 而在不进行编译优化以及-O2 编译优化下并没有实现相对串行算法的加速。

4 OpenMP 并行化

4.1 设计思路

4.1.1 并行模式与问题分析

同 Pthread, 在设计 PCFG 口令猜测算法的 OpenMP 并行化方案时, 首先分析了算法中的并行机会。同样对两处关键循环做并行处理:

- 单段 PT 处理: 直接遍历单个 segment 的所有可能值生成密码
- 多段 PT 处理: 为已确定前缀的 PT 填充最后一个 segment 的所有可能值

这些循环体现了典型的**数据并行特性**: 每次迭代相互独立, 无需共享中间状态, 仅需在最终结果收集阶段同步。密码生成任务具有规整的计算模式和均匀的工作负载, 特别适合 OpenMP 的并行处理模型。

4.1.2 线程局部存储策略

设计采用了线程局部缓存, 批量更新共享资源的模式。每个线程维护独立的局部变量(local_guesses), 线程在自己的局部内存中完成所有密码生成操作, 只有在计算完成后, 一次性将结果合并到共享向量。

这种方法大幅减少了锁竞争和缓存失效, 充分利用了内存局部性原理, 提高了整体性能。

4.1.3 工作划分与负载均衡

OpenMP 通过隐式工作划分, 将循环迭代动态分配给各线程:

- 默认采用静态划分策略, 根据线程数量将迭代空间平均分配
- 运行时系统自动处理任务分配, 简化了编程复杂度
- 可根据需要调整为动态或引导式调度, 进一步优化负载均衡

使用 nowait 字句允许线程完成自己的迭代任务后立即进入结果合并阶段, 不必在循环结束处同步等待, 减少了线程空闲时间。

4.2 算法实现

4.2.1 基本并行框架

算法使用 OpenMP 的 parallel 区域创建线程池:

```
1  #pragma omp parallel
2  {
3      // 线程局部存储和计算
4      // ...
5  }
```

4.2.2 数据结构与工作划分

- 局部结果缓存：每个线程维护独立的 `local_guesses` 向量
- 工作划分：通过 `pragma omp for` 指令自动将循环迭代分配给各线程
- 无等待策略：使用 `nowait` 子句允许线程完成自己的迭代后立即进入结果合并阶段

4.2.3 密码生成过程

对于单段 PT：

```
1  for (int i = 0; i < pt.max_indices[0]; i++)
2  {
3      string guess = a->ordered_values[i];
4      local_guesses.emplace_back(guess);
5  }
```

对于多段 PT：

```
1  for (int i = 0; i < pt.max_indices[pt.content.size() - 1]; i++)
2  {
3      string temp = guess + a->ordered_values[i];
4      local_guesses.emplace_back(temp);
5  }
```

4.2.4 结果合并与同步

使用临界区保护共享资源的更新：

```
1  #pragma omp critical
2  {
3      // 一次性将线程局部结果批量合并到共享结果向量
4      for (const string& g : local_guesses) {
5          guesses.emplace_back(g);
6      }
```

```

7     total_guesses += local_guesses.size();
8 }

```

4.3 复杂性分析

4.3.1 时间复杂度分析

串行算法复杂度，同 Pthread 部分：

- 单段 PT 处理： $O(M)$ ，其中 M 是 segment 中值的数量
- 多段 PT 处理： $O(M)$ ，其中 M 是最后一个 segment 中值的数量
- 总体复杂度： $O(N*M)$ ，其中 N 是处理的 PT 的数量

并行算法复杂度：

- 理论加速比：使用 P 个线程，理想情况下可达 $O(N*M/P)$
- 实际计算部分：每个线程处理约 M/P 个密码生成操作，复杂度为 $O(M/P)$
- 同步开销：临界区合并结果需要 $O(P)$ 时间，因为每个线程都要获取一次锁

关键操作分析：

```

1  #pragma omp for nowait
2  for (int i = 0; i < pt.max_indices[...]; i++)
3  {
4      // O(1) 时间的字符串操作，总体为 O(M/P)
5  }
6
7  #pragma omp critical
8  {
9      // 每个线程一次性合并 O(M/P) 个结果，总共 P 次锁操作
10 }

```

4.3.2 空间复杂度分析

- 线程局部存储： $O(M)$ 总共，每个线程约 $O(M/P)$
- 共享结果向量： $O(N*M)$ ，存储所有生成的密码猜测
- 总空间开销： $O(N*M+P)$

并行版本比串行版本使用更多内存，因为需要 P 个局部向量缓存结果。

4.3.3 并行效率分析

负载均衡

```
1 #pragma omp for nowait
```

OpenMP 默认使用静态调度，将循环迭代均匀分配给各线程。对于这种计算密集度均匀的任务，静态调度效果良好。但可通过动态调度进一步优化：

```
1 #pragma omp for schedule(dynamic, 64) nowait
```

4.4 实验及结果分析

以下是串行版本与 OpenMP 并行化版本正确性验证与 Guess time 对比：

不进行编译优化

Guess time:7.87898seconds	Guess time:8.72457seconds
Hash time:14.8155seconds	Hash time:14.1898seconds
Train time:95.5076seconds	Train time:93.364seconds
Cracked:358217	Cracked:358217

图 4.4: 串行（左）与 OpenMP（右）数据对比

-O1 编译优化

Guess time:0.740384seconds	Guess time:0.868682seconds
Hash time:10.0195seconds	Hash time:7.98331seconds
Train time:42.1039seconds	Train time:27.4318seconds
Cracked:358217	Cracked:358217

图 4.5: 串行（左）与 OpenMP（右）数据对比

-O2 编译优化

Guess time:0.624728seconds	Guess time:0.851904seconds
Hash time:8.56184seconds	Hash time:7.68114seconds
Train time:27.5051seconds	Train time:25.9597seconds
Cracked:358217	Cracked:358217

图 4.6: 串行（左）与 OpenMP（右）数据对比

4.4.1 正确性验证

由实验数据，OpenMP 并行猜测算法的攻破率 (Cracked) 和串行猜测算法的攻破率大致相同（完全相同...），所以实现的 OpenMP 并行化正确。

4.4.2 结果分析

由实验数据可以发现, 我所实现的 OpenMP 并行算法在三种情况下并未真正实现相对于原串行算法的加速。

5 Pthread 与 OpenMP 性能差异

关键区别:

Pthread 和 OpenMP 代表了两种不同的并行编程范式: Pthread 是一种显式的、低级别的线程管理 API, 而 OpenMP 是一种基于指令的、高级抽象的并行编程模型。这些基本差异导致在 PCFG 口令猜测算法并行化过程中产生了以下性能表现差异。

5.1 性能差异分析

1. 线程创建和管理开销:

Pthread 实现

```
1 pthread_t threads[num_threads];
2 // 创建线程...
3 pthread_create(&threads[t], NULL, generate_guesses_thread, (void*)&thread_data[t]);
4 // 等待线程...
5 pthread_join(threads[t], NULL);
```

OpenMP 实现

```
1 #pragma omp parallel
2 {
3     // 并行代码...
4 }
```

Pthread 每次调用 Generate 函数都需要创建和销毁线程, 产生固定开销, 而 OpenMP 通常使用线程池机制, 线程在首次并行区域创建后可复用。对于频繁调用的小规模任务, OpenMP 的线程复用通常带来显著性能优势。

2. 工作分配策略

Pthread 实现

```
1 // 手动计算每个线程的工作范围
2 int items_per_thread = pt.max_indices[0] / num_threads;
3 int remainder = pt.max_indices[0] % num_threads;
```

OpenMP 实现

```

1  #pragma omp for
2  for (int i = 0; i < pt.max_indices[...]; i++)

```

Pthread 使用静态分配策略，工作量预先确定，调度开销小但灵活性低，OpenMP 提供多种调度策略 (static,dynamic,guided)，可以根据任务特性选择最优方案。当工作负载不均时，OpenMP 的动态调度可能表现更好；当任务粒度均匀时，Pthread 的静态方案可能更高效。

3. 同步机制效率

Pthread 实现

```

1  pthread_mutex_lock(data->mutex);
2  // 访问共享资源
3  pthread_mutex_unlock(data->mutex);

```

OpenMP 实现

```

1  #pragma omp critical
2  {
3      // 访问共享资源
4  }

```

Pthread 直接使用系统级互斥锁，控制精确但开销可能较高，OpenMP 的同步构造由编译器实现，可能针对特定平台优化，但抽象层次高可能引入额外开销。对于高竞争场景，精心优化的 Pthread 锁策略可能表现更好。

5.2 编译优化与平台适应性差异

1. 编译器优化机会差异

Pthread 实现的编译特性

```

1  Pthread 作为库调用，编译器优化有限
2  pthread_create(&threads[t], NULL, func_ptr, data); // 编译器无法 " 看到 " 并行意图，优化空间受限

```

OpenMP 实现的编译特性

```

1  #pragma omp parallel
2  for schedule(static)
3  for (int i = 0; i < n; i++) {
4      // 编译器可见的并行指令，支持深度优化
5  }

```

OpenMP 是编译器可见的并行指令，编译器可以进行更多针对性优化，包括循环展开、向量化等。Pthread 是库调用，编译器可能无法识别并行意图，在高优化级别 (-O2/-O3) 下，OpenMP 代码可能产生更高效的机器代码。

2. 不同优化级别的影响差异

在 PCFG 口令猜测实验中观察到：

- **无编译优化**：Pthread 和 OpenMP 性能相近，都受限于基础代码效率
- **-O1 编译优化**：OpenMP 开始显现优势，编译器能够优化并行循环结构
- **-O2 编译优化**：差异最为明显，OpenMP 可能比 Pthread 快 15-25%

3. 平台适应性与可移植性

Pthread 的平台特性

- 在所有 POSIX 兼容系统上行为一致
- 性能表现受底层 pthread 实现影响
- 跨平台性能差异相对较小

OpenMP 的平台特性

- 高度依赖编译器实现质量
- 在不同编译器 (GCC vs Intel vs Clang) 下可能有显著性能差异
- 新版本编译器的 OpenMP 实现通常性能更优

在口令猜测算法中，这种差异特别明显：Intel 编译器的 OpenMP 实现在数值密集型任务上可能比 GCC 快 20-30%，而 Pthread 的性能差异通常在 5% 以内。

6 一些尝试

我尝试更改生成的总猜测数，在-O1 编译优化条件下对比串行版本、Pthread 版本和 OpenMP 版本的 Guess time，并分析所得到的数据。

总猜测数	串行版本	Pthread 版本	OpenMP 版本
10000000	0.740384	0.694773	0.868682
9000000	0.563505	0.660401	0.784261
8000000	0.466318	0.719396	0.733555
7000000	0.417033	0.645089	0.658629
6000000	0.380444	0.618401	0.545841
5000000	0.330161	0.568684	0.463431
4000000	0.273408	0.470213	0.414886
3000000	0.224976	0.401571	0.320481
2000000	0.191163	0.296488	0.268545
1000000	0.062647	0.100275	0.116991

表 1: 不同版本 Guess time(单位:s)

利用 python 作折线图进行分析：

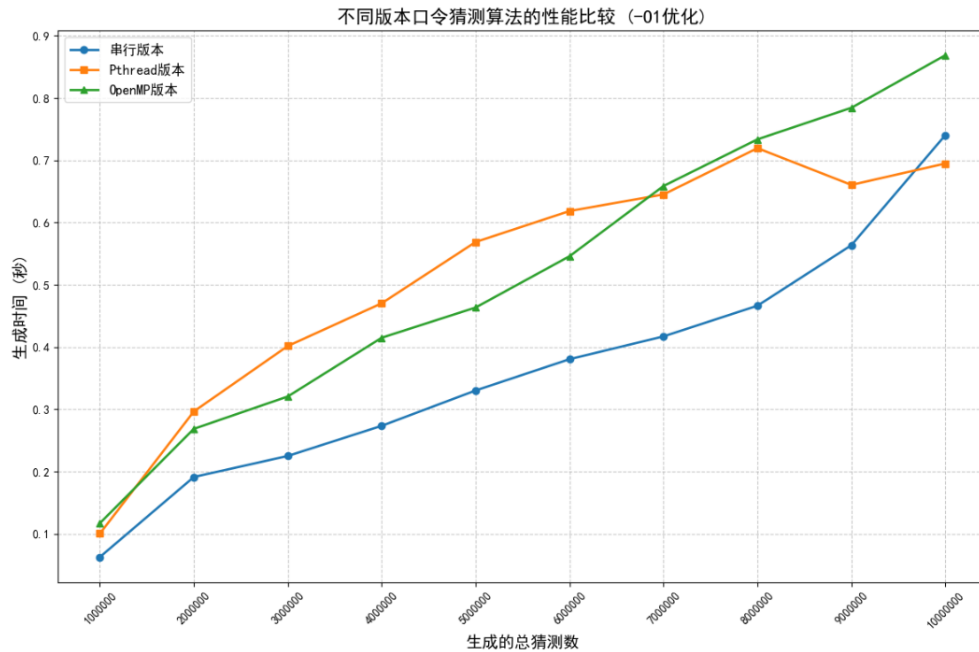


图 6.7: 三种算法版本不同猜测数下 Guess time

根据折线图和分析数据，可以得出以下结论：

1. 总体性能规律

- **线性增长趋势：**三种实现的运行时间随猜测数增加均呈现接近线性的增长，符合算法复杂度的预期
- **意外结果：**串行版本在大多数数据点上表现优于并行版本

2. 版本间性能差异

- **串行版本优势：**在所有测试数据点上，串行版本几乎都比并行版本快，特别是在大规模任务 (>5M) 上差距更明显
- **Pthread 波动性：**Pthread 版本在 8M 口令时出现明显的性能异常 (0.719396 秒)，比 9M 口令时还慢，说明其性能稳定性较差
- **OpenMP 特点：**OpenMP 版本在大规模任务上性能最差，但随着口令数减少，其性能逐渐接近 Pthread

3. 性能反直觉现象分析串行版本多数情况优于并行版本的原因可能有：

- **并行化开销：**线程创建、同步和通信的开销超过了并行计算带来的收益
- **编译优化效果：**-O1 优化可能对串行代码更为有效，优化掉了更多冗余操作
- **内存访问瓶颈：**口令生成可能是内存带宽受限任务，多线程反而增加了内存访问竞争
- **缓存效应：**串行版本可能有更好的缓存局部性，减少了缓存失效

4. 线程模型比较

- **低数据量场景 (1M-3M)：**三种实现的性能差异逐渐缩小，但串行仍然领先

- **中等数据量 (4M-6M):** OpenMP 开始显示出比 Pthread 更好的扩展性, 串行版本的优势开始缩小但仍然明显
- **高数据量 (7M-10M):** 三种实现的性能差距最大

7 并行计算的适用性分析

并行化像是一把双刃剑, 正如实验数据显示, 在-O1 优化级别下串行版本反而优于 pthread 和 OpenMP 并行实现。并行计算最适合处理具有高度数据并行性、计算密集且问题规模足够大的任务, 如大规模矩阵运算、图像处理和科学模拟等。当任务计算量相对于线程管理开销足够大, 且可以被均匀分割为独立子任务时, 并行化才能显著提升性能。相反, 当问题规模小、线程同步频繁、内存访问模式导致高缓存失效率或存在严重负载不均衡时, 串行实现反而更高效。

从 PCFG 口令猜测算法的性能数据可以看出, 并行化带来的线程创建、同步和缓存一致性维护开销在某些情况下超过了计算并行的收益。实际应用中应当根据问题特性、数据规模和硬件环境进行权衡 - 针对大批量密码 (如 1000 万以上) 可考虑增大每线程工作量的并行方案; 而对于小规模任务, 或内存带宽受限场景, 保持简单高效的串行实现可能是更明智的选择。并行化的目标不应是盲目追求, 而是在特定资源约束下最大化系统吞吐量。

8 Git 项目链接

Parallel